

CareerBrownie — Complete Software Engineering Handbook

Version 1.0 | June 2026 *AI-Powered Career Guidance Platform for Indian Students*

Table of Contents

- Part 1: Project Overview
 - Part 2: Complete Folder Structure
 - Part 3: Frontend Architecture
 - Part 4: Backend Architecture
 - Part 5: Database Handbook
 - Part 6: Database Management
 - Part 7: Authentication
 - Part 8: CRM Architecture
 - Part 9: Lead Management
 - Part 10: AI Module
 - Part 11: RAG Architecture
 - Part 12: Deployment
 - Part 13: Adding New Features
 - Part 14: Testing
 - Part 15: Security
 - Part 16: Scaling Strategy
 - Part 17: Complete User Flow Diagrams
 - Part 18: Code Explanation
-

Part 1: Project Overview

1.1 Vision

CareerBrownie is an AI-powered career guidance platform designed specifically for Indian students navigating the complex landscape of higher education and career choices. The platform combines a conversational AI chatbot, structured career recommendations, college information, scholarship discovery, and human counsellor support into a single unified experience.

Mission: Make world-class career counselling accessible to every Indian student, regardless of location or economic background, by combining AI efficiency with human expertise.

1.2 Problem Statement

Indian students face a unique set of challenges when making career decisions:

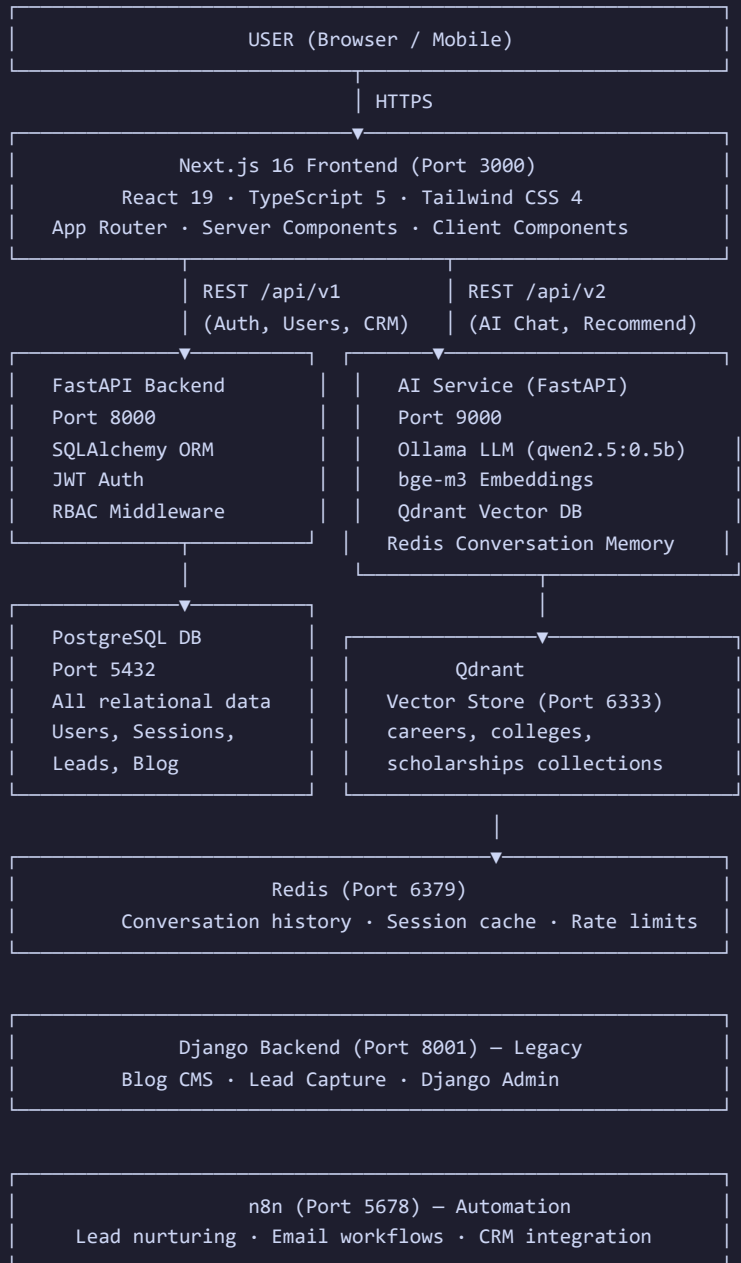
- 1. **Information overload** — Over 1,000 engineering colleges, 800+ universities, 200+ entrance exams, and thousands of career paths exist. Students cannot navigate this alone.
- 2. **Geographical disparity** — 72% of India's population lives in rural areas with no access to professional career counsellors (who are concentrated in metro cities).
- 3. **Counsellor quality variance** — School counsellors often lack updated information about emerging careers in technology, healthcare, and sustainability.
- 4. **Cost barrier** — Professional career counselling costs ₹5,000–₹50,000 per session, unaffordable for most families.
- 5. **Language barrier** — English-only resources exclude students from non-English-medium backgrounds.

CareerBrownie solves this with an AI that provides personalised, contextually accurate, always-available career guidance at near-zero marginal cost.

1.3 Target Users

User Type	Description	Primary Goal
Students	Class 10–12, undergrad, postgrad	Discover careers, find colleges, get counselling
Parents	Age 35–55, decision influencers	Understand career options, validate choices
Counsellors	Platform partners, career coaches	Manage student sessions, track progress
Admins	Internal team	Manage users, monitor leads, configure platform

1.4 High-Level Architecture



1.5 Request Lifecycle

When a student sends a chat message, here is what happens across the system in sequence:

1. Student types "What careers suit someone who loves math?" in the ChatWidget component
2. ChatWidget.sendMessage() runs:
 POST http://localhost:9000/api/v2/chat/
 Body: {message, session_id, user_id}
3. AI Service receives the request at /api/v2/chat/
4. Redis lookup: Load conversation history for session_id
 → First message: history = []
5. Intent detection: "career" keyword → intent = "career_guidance"
6. Query rewriting: LLM generates 3 search-optimized versions of the original question
7. Hybrid retrieval from Qdrant "careers" collection:
 - Dense vector search (semantic similarity)
 - Sparse BM25 search (keyword matching)
 - RRF fusion combines both result sets
8. Reranking: bge-reranker-v2-m3 cross-encoder scores top 10
 → Selects top 3 most relevant career documents
9. LLM generation: qwen2.5:0.5b generates answer using retrieved career documents as context
10. Response saved to Redis with 24h TTL
11. JSON response returned to frontend:
 {reply: "Based on your love for math...", agent_data: {careers: [...]}}
12. ChatWidget renders markdown response + career cards

Total time: 3-8 seconds (dominated by LLM generation)

1.6 Functional Requirements

Feature	Status	Priority
AI career guidance chatbot	✅ Done	P0
Structured career recommendations	✅ Done	P0
User registration / login	✅ Done	P0
JWT authentication	✅ Done	P0
Role-based access (student/counsellor/admin)	✅ Done	P0
Admin dashboard	✅ Done	P1
Lead capture form	✅ Done	P1
Blog CMS	✅ Done	P1

n8n automation workflows	✅ Done	P1
Counsellor session management	🚧 In progress	P1
Knowledge base (careers, colleges)	❌ Needs seeding	P0
Payment integration	❌ Not built	P2
Mobile app	❌ Not built	P3

1.7 Non-Functional Requirements

Requirement	Target	Current
AI response latency	< 10 seconds	3–8 seconds
API response time (non-AI)	< 500ms	~100ms
Availability	99.5% uptime	Dev (single node)
Concurrent users	100 (current)	Docker single-node
Data security	HTTPS, JWT, bcrypt	✅ Implemented
SEO	SSR for public pages	✅ Next.js SSR

Part 2: Complete Folder Structure

```

MargVedA/
└─ margveda/                                ← Project root
  ├── docker-compose.yml                    ← Local dev orchestration
  ├── docker-compose.prod.yml              ← Production orchestration
  ├── .env                                  ← Compose-level env vars (gitignored)
  ├── nginx.prod.conf                      ← Nginx reverse proxy config
  ├── README.md                            ← Project overview
  ├── START.md                             ← Quick start guide
  │
  └─ frontend-next/                        ← Next.js 16 frontend
    ├── src/
    │   ├── app/                          ← Next.js App Router
    │   │   ├── layout.tsx                ← Root layout (AuthProvider, FloatingChat)
    │   │   ├── page.tsx                  ← Landing page (/)
    │   │   ├── globals.css               ← Tailwind + custom CSS
    │   │   └─ (auth)/                    ← Route group – login, register
    │   │       ├── layout.tsx
    │   │       ├── login/page.tsx
    │   │       └─ register/page.tsx
    │   └─ (dashboard)/                  ← Route group – protected pages
    │       ├── layout.tsx
    │       ├── admin/dashboard/page.tsx
    │       ├── student/dashboard/page.tsx
    │       └─ counsellor/dashboard/page.tsx
    │   ├── blog/
    │   │   ├── page.tsx                  ← Blog listing
    │   │   └─ [slug]/page.tsx           ← Blog post
    │   ├── services/
    │   │   ├── page.tsx
    │   │   └─ ai-career-guidance/page.tsx
    │   ├── about/page.tsx
    │   ├── contact/page.tsx
    │   ├── enterprise/page.tsx
    │   ├── faq/page.tsx
    │   ├── privacy-policy/page.tsx
    │   └─ refund-policy/page.tsx
    │   ├── components/
    │   │   ├── chat/
    │   │   │   ├── ChatWidget.tsx        ← Main chat interface
    │   │   │   └─ FloatingChatButton.tsx ← Floating button
    │   │   └─ ui/                        ← Shared UI components
    │   └─ layout/
    │       ├── Navbar.tsx
    │       └─ Footer.tsx
    │       └─ dashboard/
    │   ├── contexts/
    │   │   └─ AuthContext.tsx            ← Global auth state
    │   ├── lib/
    │   │   └─ api.ts                     ← Axios instance + interceptors
    │   ├── types/
    │   │   └─ index.ts                   ← TypeScript type definitions
    │   ├── public/                       ← Static files
    │   │   ├── favicon.ico
    │   │   ├── logo.svg
    │   │   └─ images/
    │   ├── next.config.ts
    │   ├── tailwind.config.ts
    │   ├── tsconfig.json
    │   ├── package.json
    │   └─ .env.local                     ← Frontend env vars (gitignored)
  
```

```

├── backend/                                ← FastAPI backend
│   ├── app/
│   │   ├── main.py                        ← App factory, CORS, router registration
│   │   ├── database.py                   ← SQLALchemy engine and session
│   │   ├── core/
│   │   │   ├── config.py                 ← Pydantic Settings
│   │   │   ├── security.py               ← JWT, bcrypt
│   │   │   └── logging.py                ← Structured logging setup
│   │   ├── models/                      ← SQLALchemy ORM models
│   │   │   ├── user.py
│   │   │   ├── recommendation.py
│   │   │   ├── session.py
│   │   │   └── lead.py
│   │   ├── routers/                     ← FastAPI route handlers
│   │   │   ├── auth.py
│   │   │   ├── users.py
│   │   │   ├── recommendations.py
│   │   │   ├── admin.py
│   │   │   └── leads.py
│   │   ├── services/                    ← Business logic layer
│   │   │   ├── recommendation_service.py
│   │   │   └── user_service.py
│   │   ├── dependencies/                ← FastAPI dependency injection
│   │   │   ├── auth.py                  ← get_current_user, require_roles
│   │   └── schemas/                     ← Pydantic request/response schemas
│   │       ├── auth.py
│   │       └── user.py
│   ├── alembic/                         ← Database migrations
│   │   ├── versions/
│   │   └── env.py
│   ├── tests/
│   │   ├── conftest.py
│   │   ├── test_smoke.py
│   │   └── test_regression.py
│   ├── Dockerfile
│   ├── requirements.txt
│   └── .env.example
├── ai_service/                            ← AI microservice (FastAPI)
│   ├── app/
│   │   ├── main.py                      ← App factory, startup, health endpoint
│   │   ├── core/
│   │   │   ├── config.py                ← AI service settings
│   │   │   └── logging.py
│   │   ├── chatbot/
│   │   │   ├── engine.py                ← Main RAG pipeline orchestrator
│   │   │   ├── bot.py                   ← FastAPI router for /api/v2/chat/
│   │   │   └── memory.py                 ← Redis conversation history
│   │   ├── recommender/
│   │   │   └── career_recommender.py     ← Structured recommendations
│   │   ├── llm/
│   │   │   ├── base.py                  ← Abstract LLM interface
│   │   │   └── ollama.py                 ← OllamaLLM + AnthropicFallback
│   │   ├── embeddings/
│   │   │   └── bge_m3.py                 ← BAAI/bge-m3 wrapper
│   │   ├── retrieval/
│   │   │   ├── qdrant.py                ← Qdrant client wrapper
│   │   │   └── reranker.py               ← bge-reranker-v2-m3 wrapper
│   │   └── agents/
│   │       └── orchestrator.py           ← Multi-agent coordination

```



```
|
| ├── Dockerfile
| ├── requirements.txt
| └── .env.example
|
| ├── backend_django/           ← Django backend (legacy CMS)
| | ├── blog/                  ← Blog app
| | | ├── models.py
| | | ├── views.py
| | | ├── urls.py
| | | └── migrations/
| | ├── leads/                 ← Lead capture app
| | | ├── models.py
| | | └── views.py
| | ├── users/                 ← User management app
| | | └── views.py
| | ├── ai_engine/             ← Django → AI service bridge
| | | ├── views.py
| | | └── urls.py
| | └── manage.py
|
| └── frontend/                 ← Legacy Reflex frontend (deprecated)
```

Part 3: Frontend Architecture

3.1 Technology Choices

Technology	Version	Why
Next.js	16.2.9	App Router, SSR/SSG for SEO, file-based routing
React	19	Concurrent features, improved performance
TypeScript	5	Type safety, better IDE support
Tailwind CSS	4	Utility-first, rapid UI development
Axios	Latest	HTTP client with interceptors for auth

3.2 App Router Structure

Next.js App Router uses the file system for routing. Every `page.tsx` file inside `src/app/` becomes a route.

```
src/app/
├── page.tsx           → /
├── layout.tsx         → wraps all pages
├── (auth)/            → route group (no URL segment)
│   ├── login/page.tsx → /login
│   └── register/page.tsx → /register
├── (dashboard)/       → route group (no URL segment)
│   ├── layout.tsx     → dashboard shell with sidebar
│   ├── admin/dashboard/page.tsx → /admin/dashboard
│   ├── student/dashboard/page.tsx → /student/dashboard
│   └── counsellor/dashboard/page.tsx → /counsellor/dashboard
└── blog/
    ├── page.tsx       → /blog
    └── [slug]/page.tsx → /blog/best-careers-for-math-students
```

Route Groups (`(auth)` , `(dashboard)`) are a Next.js feature that allows you to group routes without affecting the URL. The `(auth)` group's layout adds the auth card styling without adding `/auth/` to the URL.

3.3 Root Layout

```
// src/app/layout.tsx
export const metadata: Metadata = {
  title: {
    default: "CareerBrownie – AI Career Guidance",
    template: "%s | CareerBrownie",
  },
  description: "AI-powered career counselling for Indian students",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <AuthProvider>
          {children}
          <FloatingChatButton />
        </AuthProvider>
      </body>
    </html>
  );
}
```

The `metadata.template` setting means if a page exports `export const metadata = { title: "Blog" }`, the browser tab shows "Blog | CareerBrownie".

3.4 Authentication Context

```
// src/contexts/AuthContext.tsx
interface AuthContextType {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;
  login: (token: string, user: User) => void;
  logout: () => void;
}

export function AuthProvider({ children }) {
  const [user, setUser] = useState<User | null>(null);
  const [token, setToken] = useState<string | null>(null);

  useEffect(() => {
    // Restore session from localStorage on page load
    const savedToken = localStorage.getItem("token");
    const savedUser = localStorage.getItem("user");
    if (savedToken && savedUser) {
      setToken(savedToken);
      setUser(JSON.parse(savedUser));
      api.defaults.headers.Authorization = `Bearer ${savedToken}`;
    }
  }, []);

  const login = (newToken: string, newUser: User) => {
    setToken(newToken);
    setUser(newUser);
    localStorage.setItem("token", newToken);
    localStorage.setItem("user", JSON.stringify(newUser));
    api.defaults.headers.Authorization = `Bearer ${newToken}`;
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    delete api.defaults.headers.Authorization;
  };

  return (
    <AuthContext.Provider value={{ user, token, isAuthenticated: !!user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

3.5 HTTP Client

```
// src/lib/api.ts
import axios from "axios";

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  timeout: 30000,
});

// Attach JWT to every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Handle 401 globally - redirect to login
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.clear();
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default api;
```

3.6 ChatWidget Component

The ChatWidget is the core of the product. It handles:

- Conversation display (user and AI messages)
- Input field and send button
- Loading indicator during AI processing
- Markdown rendering of AI responses
- Career card display from `agent_data`

```

const sendMessage = async () => {
  if (!input.trim() || isLoading) return;

  const userMsg: Message = { role: "user", content: input };
  setMessages(prev => [...prev, userMsg]);
  setInput("");
  setIsLoading(true);

  try {
    const response = await fetch(
      `${process.env.NEXT_PUBLIC_AI_URL}/api/v2/chat/`,
      {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          message: input,
          session_id: sessionId,
          user_id: user?.id,
        }),
      }
    );

    const data = await response.json();
    setMessages(prev => [...prev, { role: "assistant", content: data.reply }]);

    if (data.agent_data?.careers) {
      setCareerCards(data.agent_data.careers);
    }
  } catch {
    setMessages(prev => [...prev, {
      role: "assistant",
      content: "I'm having trouble connecting. Please try again.",
    }]);
  } finally {
    setIsLoading(false);
  }
};

```

3.7 FloatingChatButton — SSR Consideration

```
// Lazy load ChatWidget - it uses sessionStorage which doesn't exist on server
const ChatWidget = dynamic(() => import("./ChatWidget"), { ssr: false });

export default function FloatingChatButton() {
  const [open, setOpen] = useState(false);

  return (
    <>
      <button
        onClick={() => setOpen(!open)}
        className="fixed bottom-6 right-6 z-50 rounded-full bg-purple-600 p-4 shadow-lg"
        aria-label="Open chat"
      >
        {open ? <XIcon /> : <ChatIcon />}
      </button>
      {open && <ChatWidget />}
    </>
  );
}
```

`ssr: false` ensures ChatWidget is never rendered on the server. Without this, Next.js SSR would crash trying to access `sessionStorage` .

Part 4: Backend Architecture

4.1 Three-Layer Architecture

```
HTTP Request
↓
Router (app/routers/*.py)
  → Input validation (Pydantic schemas)
  → Authentication (Depends)
↓
Service (app/services/*.py)
  → Business logic
  → External service calls
↓
Model (app/models/*.py)
  → Database queries via SQLAlchemy
↓
PostgreSQL
```

This separation means:

- **Routers** only handle HTTP concerns (request parsing, response formatting)
- **Services** contain reusable business logic (can be called from multiple routes)
- **Models** are only concerned with data structure and persistence

4.2 Application Factory

```
# backend/app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.database import engine, Base
from app.routers import auth, users, recommendations, admin, leads

app = FastAPI(
    title=settings.PROJECT_NAME,
    version=settings.VERSION,
    docs_url="/docs",
    redoc_url="/redoc",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.BACKEND_CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router, prefix="/api/v1/auth", tags=["Authentication"])
app.include_router(users.router, prefix="/api/v1/users", tags=["Users"])
app.include_router(recommendations.router, prefix="/api/v1/recommendations")
app.include_router(admin.router, prefix="/api/v1/admin", tags=["Admin"])
app.include_router(leads.router, prefix="/api/v1/leads", tags=["Leads"])

@app.on_event("startup")
async def startup():
    Base.metadata.create_all(bind=engine)

@app.get("/health")
def health():
    return {"status": "ok", "version": settings.VERSION}
```

4.3 Complete API Reference

Authentication Endpoints

Method	URL	Auth	Description
POST	<code>/api/v1/auth/register</code>	None	Create new account
POST	<code>/api/v1/auth/login</code>	None	Login, get JWT
GET	<code>/api/v1/auth/me</code>	Bearer	Get current user profile
POST	<code>/api/v1/auth/refresh</code>	Bearer	Refresh JWT token

User Endpoints

Method	URL	Auth	Description
GET	<code>/api/v1/users/profile</code>	Bearer	Get own profile
PATCH	<code>/api/v1/users/profile</code>	Bearer	Update profile

Recommendation Endpoints

Method	URL	Auth	Description
POST	<code>/api/v1/recommendations</code>	Bearer	Generate career recommendations
GET	<code>/api/v1/recommendations</code>	Bearer	Get saved recommendations

Admin Endpoints

Method	URL	Auth	Role
GET	<code>/api/v1/admin/stats</code>	Bearer	admin
GET	<code>/api/v1/admin/users</code>	Bearer	admin
PATCH	<code>/api/v1/admin/users/{id}/role</code>	Bearer	admin
GET	<code>/api/v1/admin/leads</code>	Bearer	admin

Lead Endpoints

Method	URL	Auth	Description
POST	<code>/api/v1/leads</code>	None	Capture lead from landing page
GET	<code>/api/v1/leads</code>	Bearer (admin)	List all leads

AI Service Endpoints

Method	URL	Auth	Description
POST	<code>/api/v2/chat/</code>	X-Internal-API-Key	Chat message
POST	<code>/api/v1/recommend</code>	X-Internal-API-Key	Career recommendations
GET	<code>/health</code>	None	Health check

4.4 Dependency Injection Pattern

FastAPI uses `Depends()` for dependency injection — a way to declare shared logic that runs before your route handler.

```
# dependencies/auth.py
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    payload = decode_token(token)
    user_id = int(payload.get("sub"))
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=401, detail="User not found")
    return user

def require_roles(allowed_roles: list[str]):
    def checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role.value not in allowed_roles:
            raise HTTPException(status_code=403, detail="Insufficient permissions")
        return current_user
    return checker

# Usage in route:
@router.get("/admin/users")
def list_users(admin: User = Depends(require_roles(["admin"])), db: Session = Depends(get_db)):
    return db.query(User).all()
```

Part 5: Database Handbook

5.1 Database Technology

CareerBrownie uses **PostgreSQL 16** as the primary relational database. SQLAlchemy provides the Python ORM layer; Alembic handles schema migrations.

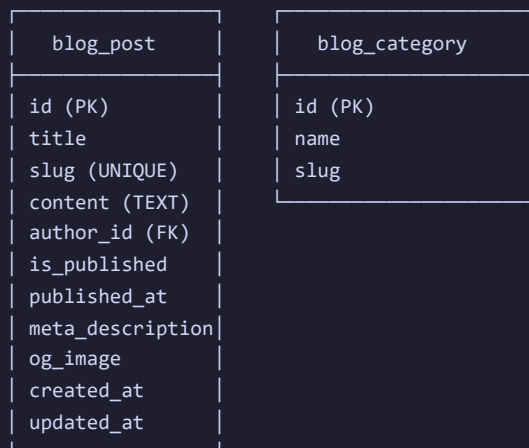
Why PostgreSQL over MySQL or SQLite:

- Full ACID compliance with better concurrency under write loads
- Native JSON/JSONB columns (useful for storing AI metadata)
- Full-text search built in
- Superior indexing options (partial indexes, BRIN for time-series)
- First-class support in all cloud providers

5.2 Entity Relationship Diagram



Django Database (separate PostgreSQL or shared schema):



5.3 All Tables — SQL Definitions

Users Table

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(200) NOT NULL,  
  email VARCHAR(254) UNIQUE NOT NULL,  
  hashed_password VARCHAR(100) NOT NULL,  
  phone VARCHAR(20),  
  role VARCHAR(20) NOT NULL DEFAULT 'student',  
    -- Allowed: 'student', 'counsellor', 'admin'  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_role ON users(role);
```

Recommendations Table

```
CREATE TABLE recommendations (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  career_title VARCHAR(200) NOT NULL,  
  career_description TEXT,  
  match_score FLOAT DEFAULT 0.0,  
  metadata JSONB, -- Stores additional AI-generated metadata  
  created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_recommendations_user ON recommendations(user_id);  
CREATE INDEX idx_recommendations_score ON recommendations(match_score DESC);
```

Leads Table

```

CREATE TABLE leads (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  email VARCHAR(254) NOT NULL,
  phone VARCHAR(20),
  goal TEXT,
  budget VARCHAR(100),
  source VARCHAR(100) DEFAULT 'website',
  score INTEGER DEFAULT 0,
  status VARCHAR(50) DEFAULT 'new',
  -- Allowed: 'new', 'contacted', 'qualified', 'converted', 'lost'
  assigned_to INTEGER REFERENCES users(id),
  notes TEXT,
  created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_leads_status ON leads(status);
CREATE INDEX idx_leads_score ON leads(score DESC);
CREATE INDEX idx_leads_created ON leads(created_at DESC);

```

Counselling Sessions Table

```

CREATE TABLE counselling_sessions (
  id SERIAL PRIMARY KEY,
  student_id INTEGER NOT NULL REFERENCES users(id),
  counsellor_id INTEGER REFERENCES users(id),
  status VARCHAR(50) DEFAULT 'pending',
  -- Allowed: 'pending', 'active', 'completed', 'cancelled'
  notes TEXT,
  scheduled_at TIMESTAMP WITHOUT TIME ZONE,
  completed_at TIMESTAMP WITHOUT TIME ZONE,
  created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
);

```

5.4 SQLAlchemy Models

```

# backend/app/models/user.py
from sqlalchemy import Column, Integer, String, Boolean, DateTime, Enum
from sqlalchemy.orm import relationship
from app.database import Base
import enum

class UserRole(str, enum.Enum):
    student = "student"
    counsellor = "counsellor"
    admin = "admin"

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(200), nullable=False)
    email = Column(String(254), unique=True, nullable=False, index=True)
    hashed_password = Column(String(100), nullable=False)
    phone = Column(String(20))
    role = Column(Enum(UserRole), default=UserRole.student, nullable=False)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    recommendations = relationship("Recommendation", back_populates="user")

    def to_dict(self):
        return {
            "id": self.id,
            "name": self.name,
            "email": self.email,
            "role": self.role.value,
            "is_active": self.is_active,
            "created_at": self.created_at.isoformat(),
        }

```

5.5 Database Connection and Sessions

```
# backend/app/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from app.core.config import settings

engine = create_engine(
    settings.DATABASE_URL,
    pool_size=10,          # Keep 10 connections ready
    max_overflow=20,       # Allow 20 extra connections under load
    pool_pre_ping=True,    # Test connection before using (handles DB restarts)
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Part 6: Database Management

6.1 Connecting to the Database

```
# Via Docker
docker exec -it careerbrownie_postgres psql -U careerbrownie -d careerbrownie

# Via psql directly (if PostgreSQL client installed)
psql postgresql://careerbrownie:careerbrownie_password@localhost:5432/careerbrownie
```

6.2 Common Database Queries

View all tables

```
\dt          -- List all tables
\d users      -- Describe users table structure
```

User management

```
-- All users with roles
SELECT id, name, email, role, created_at FROM users ORDER BY created_at DESC;

-- Promote user to admin
UPDATE users SET role = 'admin' WHERE email = 'admin@careerbrownie.com';

-- Deactivate user
UPDATE users SET is_active = FALSE WHERE id = 42;
```

Lead analytics

```
-- Leads by status
SELECT status, COUNT(*) FROM leads GROUP BY status ORDER BY COUNT(*) DESC;

-- Hot leads (score > 50) from last 7 days
SELECT name, email, phone, score, created_at
FROM leads
WHERE score > 50 AND created_at > NOW() - INTERVAL '7 days'
ORDER BY score DESC;

-- Lead conversion rate
SELECT
    COUNT(*) FILTER (WHERE status = 'converted') AS converted,
    COUNT(*) AS total,
    ROUND(COUNT(*) FILTER (WHERE status = 'converted') * 100.0 / COUNT(*), 2) AS conversion_pct
FROM leads;
```

Recommendation analytics

```
-- Most recommended careers
SELECT career_title, COUNT(*) AS count, AVG(match_score) AS avg_score
FROM recommendations
GROUP BY career_title
ORDER BY count DESC
LIMIT 20;
```

6.3 Database Backup and Restore

```
# Backup (from Docker host)
docker exec careerbrownie_postgres \
    pg_dump -U careerbrownie careerbrownie \
    > backup_$(date +%Y%m%d_%H%M%S).sql

# Restore
docker exec -i careerbrownie_postgres \
    psql -U careerbrownie careerbrownie \
    < backup_20260629_120000.sql

# Backup with compression
docker exec careerbrownie_postgres \
    pg_dump -U careerbrownie -Fc careerbrownie \
    > backup.pgdump

# Restore compressed backup
docker exec -i careerbrownie_postgres \
    pg_restore -U careerbrownie -d careerbrownie \
    < backup.pgdump
```

6.4 Alembic Migrations

```
# Create a migration after changing a model
docker exec careerbrownie_backend \
    alembic revision --autogenerate -m "add_phone_column_to_users"

# Apply all pending migrations
docker exec careerbrownie_backend alembic upgrade head

# Rollback one migration
docker exec careerbrownie_backend alembic downgrade -1

# View migration history
docker exec careerbrownie_backend alembic history
```

Part 7: Authentication

7.1 JWT Deep Dive

JSON Web Tokens (JWT) are the authentication mechanism for the CareerBrownie API. A JWT is a Base64-encoded string with three parts separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  ← Header (algorithm + type)
.eyJzdWIiOiI0MiIsInJvbGUiOiJzdHVkZW50In0  ← Payload (claims)
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c  ← Signature
```

The **Header** declares the algorithm (HS256 = HMAC-SHA256).

The **Payload** contains claims:

```
{
  "sub": "42",           // Subject = user ID
  "role": "student",     // For RBAC
  "email": "user@example.com",
  "iat": 1719600000,     // Issued at (Unix timestamp)
  "exp": 1719686400      // Expires at (24 hours later)
}
```

The **Signature** is computed as:

```
HMAC_SHA256(
  base64(header) + "." + base64(payload),
  SECRET_KEY
)
```

Only someone with the `SECRET_KEY` can produce a valid signature. This means tokens cannot be forged, and the server doesn't need a database lookup to validate a token — it just verifies the signature mathematically.

7.2 Authentication Flow

1. POST /api/v1/auth/login
Body: {email, password}
2. Backend:
 - a. Query: SELECT * FROM users WHERE email = ?
 - b. bcrypt.verify(password, user.hash_password)
 - c. Create JWT: {sub: user.id, role: user.role, exp: +24h}
 - d. Return: {access_token: "eyJ...", user: {...}}
3. Frontend:
 - a. localStorage.setItem("token", access_token)
 - b. AuthContext.setUser(user)
 - c. axios.defaults.headers.Authorization = "Bearer eyJ..."
4. Subsequent requests:
Authorization: Bearer eyJ...
5. Backend validates:
 - a. Extract token from Authorization header
 - b. Verify signature with SECRET_KEY
 - c. Check exp claim not past
 - d. Return user_id from sub claim
6. Token expires (after 24 hours):
 - a. Next API call returns 401
 - b. Axios interceptor catches 401
 - c. localStorage.clear()
 - d. window.location.href = "/login"

7.3 RBAC — Role-Based Access Control

Role	Capabilities
student	Chat, view recommendations, update own profile
counsellor	All student permissions + view assigned sessions + update session notes
admin	All permissions + manage users + view all leads + system configuration

Implementation:

```
# Any authenticated user
@router.get("/profile")
def get_profile(user: User = Depends(get_current_user)):
    return user

# Counsellor or admin only
@router.get("/sessions")
def get_sessions(user: User = Depends(require_roles(["counsellor", "admin"]))):
    return sessions_for_user(user)

# Admin only
@router.delete("/users/{user_id}")
def delete_user(
    user_id: int,
    admin: User = Depends(require_roles(["admin"])),
    db: Session = Depends(get_db)
):
    db.query(User).filter(User.id == user_id).delete()
    db.commit()
```

7.4 Password Security

CareerBrownie uses **bcrypt** for password hashing with a work factor of 12 (default).

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Creating a hash takes ~100ms (intentionally slow)
hashed = pwd_context.hash("user_password_here")
# → "$2b$12$EixZaYVK1fsbw1ZfbX3OXePaWxn96p36WQoeG6Lruj3vjPGga31lW"

# Verification also takes ~100ms
is_valid = pwd_context.verify("user_password_here", hashed)
# → True
```

Why slow hashing matters: If your database is breached and all hashed passwords are stolen, an attacker with a modern GPU can test ~10 billion MD5 hashes per second. Against bcrypt work factor 12, they can test only ~200 hashes per second. A 8-character password that would be cracked in under 1 second with MD5 takes over 200 years with bcrypt.

Part 8: CRM Architecture

8.1 Overview

CareerBrownie's CRM (Customer Relationship Management) is built on top of n8n — an open-source workflow automation platform. Rather than building a custom CRM, n8n connects the lead capture system, email service, Google Sheets, and counsellor assignment logic through visual workflows.

8.2 n8n Workflow Architecture

```
Lead Created (webhook)
↓
Parse lead data
↓
Score evaluation (IF node):
  Score > 70 → "Hot Lead" path
  Score 40-70 → "Warm Lead" path
  Score < 40 → "Cold Lead" path
↓ (Hot path)
Immediate notification to counsellor (Email)
Create Google Sheets row
Assign counsellor
Set callback in 1 hour
↓ (Warm path)
Add to email drip campaign
Schedule follow-up in 24 hours
↓ (Cold path)
Add to weekly newsletter
No immediate action
```

8.3 Lead Scoring Algorithm

```
def calculate_lead_score(data: dict) -> int:
    score = 10 # Base score for any lead

    # Contact completeness
    if data.get("phone"):
        score += 20 # Phone = high intent
    if data.get("goal"):
        score += 15 # Goal = defined need
    if data.get("budget"):
        score += 10 # Budget = purchase readiness

    # Email quality
    email = data.get("email", "")
    if "@" in email and not email.endswith(("@gmail.com", "@yahoo.com")):
        score += 10 # Institutional email suggests professional

    # Source quality
    source_scores = {
        "referral": 25, # Referred leads convert better
        "google_ads": 15,
        "organic": 10,
        "social": 5,
        "website": 0,
    }
    score += source_scores.get(data.get("source", "website"), 0)

    return min(score, 100) # Cap at 100
```

Part 9: Lead Management

9.1 Lead Lifecycle



Status	Meaning	Next Action
new	Just captured	Auto-assign or manual review
contacted	Email/phone made	Wait for response
qualified	Confirmed intent and budget	Book session
converted	Paid or registered	Onboard to platform
lost	No response after 3 attempts	Archive

9.2 Lead Capture API

```
# backend/app/routers/leads.py
@router.post("/", status_code=201)
async def create_lead(data: LeadCreateRequest, db: Session = Depends(get_db)):
    # Calculate score
    score = calculate_lead_score(data.dict())

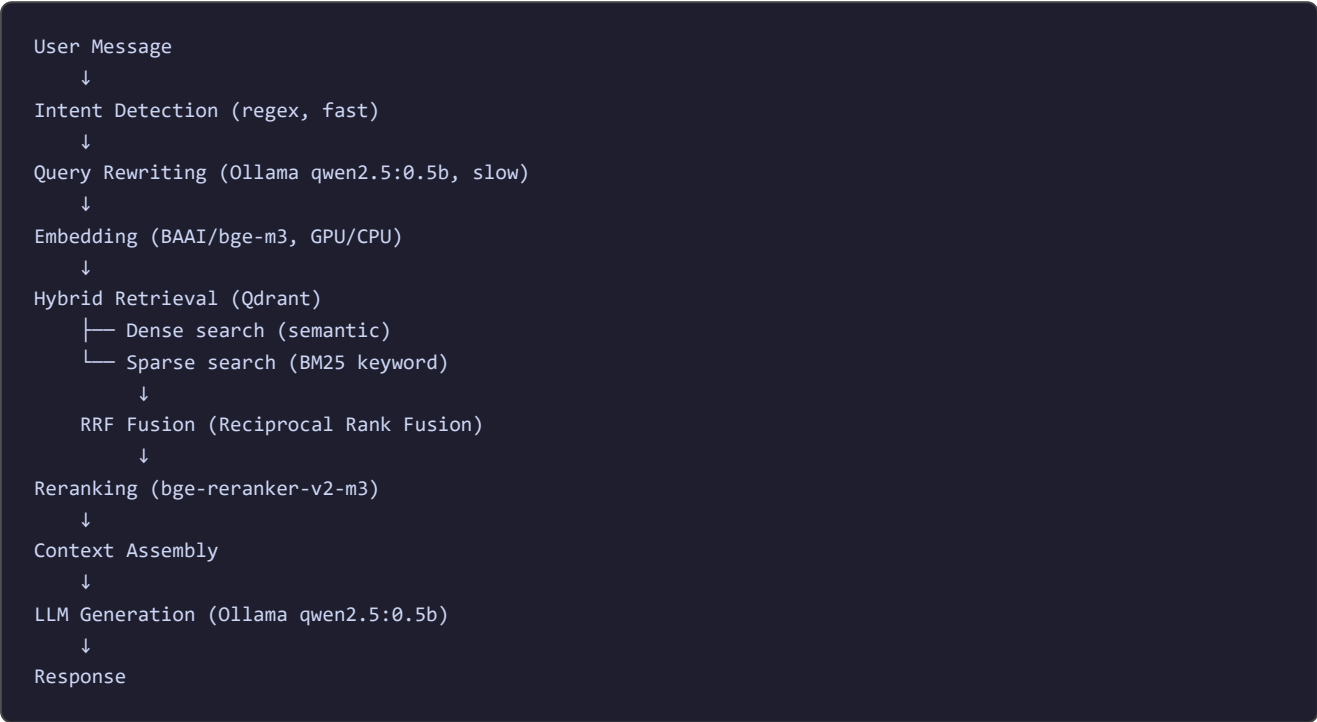
    lead = Lead(
        name=data.name,
        email=data.email,
        phone=data.phone,
        goal=data.goal,
        budget=data.budget,
        source=data.source or "website",
        score=score,
        status="new",
    )
    db.add(lead)
    db.commit()
    db.refresh(lead)

    # Trigger n8n automation (non-blocking)
    asyncio.create_task(trigger_n8n_webhook("new-lead", {
        "lead_id": lead.id,
        "name": lead.name,
        "email": lead.email,
        "score": score,
    }))

    return {"success": True, "lead_id": lead.id}
```

Part 10: AI Module

10.1 AI Stack Overview



10.2 LLM — qwen2.5:0.5b

Qwen2.5:0.5b is a 500 million parameter language model developed by Alibaba. It's the smallest variant in the Qwen 2.5 family.

Property	Value
Parameters	500M
Context window	32,768 tokens
Model size on disk	~400MB
RAM required	~1GB
Inference speed (CPU)	~150 tokens/second
Languages	English + Chinese (primarily)
License	Apache 2.0 (commercial use ok)

For production at scale, you would upgrade to `qwen2.5:7b` (7 billion parameters) for significantly better reasoning, or `llama3.1:8b` for better English.

10.3 Embeddings — BAAI/bge-m3

bge-m3 (BAAI General Embedding, Multi-lingual, Multi-granularity, Multi-functionality) is a state-of-the-art embedding model that supports:

- **Dense embeddings** — 1024-dimensional float vectors, best for semantic similarity
- **Sparse embeddings** — token-weight pairs (BM25-like), best for keyword matching
- **ColBERT** — token-level multi-vector representation (not used here)

The combination of dense + sparse enables hybrid search — you get both semantic understanding ("careers for people who like working with numbers") AND keyword matching ("mathematics career").

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel(
    "BAAI/bge-m3",
    use_fp16=True,      # Half precision — 2x faster, minimal quality loss
    device="cpu",       # "cuda" for GPU
)

result = model.encode(
    ["I love mathematics, what career should I choose?"],
    batch_size=1,
    return_dense=True,
    return_sparse=True,
)

dense_vector = result["dense_vecs"][0]    # shape: (1024,)
sparse_vector = result["lexical_weights"][0] # {word_id: weight}
```

10.4 Vector Database — Qdrant

Qdrant stores career, college, and scholarship information as vector embeddings. When a student asks a question, the question is embedded and compared against all stored documents to find the most semantically relevant ones.

Collections:

Collection	Content	Documents
<code>careers</code>	Career descriptions, requirements, salary	~200 careers
<code>colleges</code>	College profiles, fees, courses	~500 colleges
<code>scholarships</code>	Scholarship details, eligibility	~100 scholarships
<code>response_cache</code>	Previous AI responses	Dynamic

Hybrid search query:

```

from qdrant_client.models import Prefetch, FusionQuery, Fusion, SparseVector

results = await qdrant.query_points(
    collection_name="careers",
    prefetch=[
        Prefetch(
            query=dense_vector.tolist(), # float list
            using="dense",
            limit=10,
        ),
        Prefetch(
            query=SparseVector(
                indices=list(sparse_vector.keys()),
                values=list(sparse_vector.values()),
            ),
            using="sparse",
            limit=10,
        ),
    ],
    query=FusionQuery(fusion=Fusion.RRF), # Reciprocal Rank Fusion
    limit=10,
    with_payload=True,
)

```

10.5 Reciprocal Rank Fusion (RRF)

RRF combines two ranked lists into a single merged list. The formula for each document:

$$\text{RRF_score}(\text{doc}) = \sum (1 / (k + \text{rank_in_list_i}))$$

where $k = 60$ (constant preventing division-by-zero effects)

rank = position in the result list (1-indexed)

Example:

- Document A ranked #1 in dense, #5 in sparse: $\text{score} = 1/(60+1) + 1/(60+5) = 0.0164 + 0.0154 = 0.0318$
- Document B ranked #3 in dense, #2 in sparse: $\text{score} = 1/(60+3) + 1/(60+2) = 0.0159 + 0.0161 = 0.0320$

Document B wins despite not being first in either list — because it performed consistently well across both retrieval methods.

Part 11: RAG Architecture

11.1 What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that improves LLM responses by providing relevant documents as context. Instead of relying on the LLM's training data (which may be outdated or wrong), RAG pulls fresh, authoritative information and includes it in the prompt.

Without RAG:

```
User: "What is the fee for Symbiosis Institute of Business Management?"  
LLM: "The fees at SIBM are approximately ₹8-12 lakhs..." (may be outdated)
```

With RAG:

```
User: "What is the fee for SIBM?"  
Retrieved: [SIBM profile: "Total fee: ₹3.2 lakhs for 2-year MBA program, 2024-25"]  
LLM: "Based on the current information, SIBM's 2-year MBA program costs ₹3.2 lakhs total..."
```

11.2 Knowledge Ingestion Pipeline

Before the RAG system can answer questions, documents must be ingested into Qdrant. The ingestion pipeline:

```

# scripts/ingest_careers.py
import asyncio
from qdrant_client import AsyncQdrantClient
from qdrant_client.models import PointStruct
from FlagEmbedding import BGEM3FlagModel

careers = [
    {
        "name": "Software Engineer",
        "description": "Design, develop, and maintain software systems. "
            "Work with programming languages like Python, Java, or JavaScript. "
            "Typical salary: ₹6-25 LPA for freshers.",
        "tags": ["technology", "coding", "computer science", "STEM"],
        "education": ["B.Tech CSE", "BCA", "B.Sc Computer Science"],
        "salary_range": "6-25 LPA",
        "growth": "high",
    },
    # ... hundreds more careers
]

async def ingest():
    model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)
    client = AsyncQdrantClient(url="http://localhost:6333")

    for i, career in enumerate(careers):
        # Encode the career description
        text = f"{career['name']}: {career['description']}"
        result = model.encode([text], return_dense=True, return_sparse=True)

        await client.upsert(
            collection_name="careers",
            points=[PointStruct(
                id=i,
                vector={
                    "dense": result["dense_vecs"][0].tolist(),
                    "sparse": {
                        "indices": list(result["lexical_weights"][0].keys()),
                        "values": list(result["lexical_weights"][0].values()),
                    },
                },
                payload=career,
            )],
        )
        print(f"Ingested {i+1}/{len(careers)}: {career['name']}")

    asyncio.run(ingest())

```

11.3 System Prompts

The system prompt shapes all AI responses. CareerBrownie's counsellor prompt:


```
CAREER_COUNSELLOR_SYSTEM_PROMPT = """You are CareerBrownie's AI career counsellor, \
specialised in helping Indian students aged 15-25 navigate their career choices.
```

Core principles:

- Be specific about Indian context: mention JEE, NEET, CLAT, CAT cutoffs
- Give salary ranges in Indian Rupees (LPA)
- Mention real colleges and courses available in India
- Be honest about competitive exams and difficulty levels
- Encourage students; avoid discouraging language
- If asked about international colleges, clarify costs in INR equivalents
- Use simple English; avoid jargon unless explaining it

Response format:

- Lead with the direct answer
- Use bullet points for lists of careers or colleges
- Include salary range and education path
- End with an actionable next step

```
Knowledge cutoff: Use ONLY information from the provided context. \
If the context doesn't contain relevant information, say so honestly."""
```

11.4 Conversation Memory

Redis stores conversation history per session:

```
# ai_service/app/chatbot/memory.py
import json
import redis.asyncio as redis
from app.core.config import settings

HISTORY_TTL = 86400 # 24 hours

async def load_history(session_id: str) -> list[dict]:
    r = redis.from_url(settings.REDIS_URL)
    raw = await r.get(f"chat:{session_id}:history")
    if raw:
        return json.loads(raw)
    return []

async def save_history(session_id: str, history: list[dict]) -> None:
    r = redis.from_url(settings.REDIS_URL)
    await r.setex(
        f"chat:{session_id}:history",
        HISTORY_TTL,
        json.dumps(history),
    )
```

Each entry in history is `{"role": "user" | "assistant", "content": "..."} . Only the last 6 messages (3 exchanges) are sent to the LLM to keep prompts manageable.`

Part 12: Deployment

12.1 Docker Architecture

```
Docker Host (your machine or cloud VM)
├── Docker Engine
│   └── docker-compose network: careerbrownie_default
│       ├── postgres (postgres:16-alpine)
│       ├── redis (redis:7-alpine)
│       ├── qdrant (qdrant/qdrant:v1.11.3)
│       ├── ollama (ollama/ollama:latest)
│       ├── ai_service (custom image built from ./ai_service)
│       ├── backend (custom image built from ./backend)
│       ├── frontend (custom image built from ./frontend-next)
│       └── n8n (n8nio/n8n:latest)
```

All containers share a bridge network. Containers reference each other by service name (e.g., `http://postgres:5432` , `http://ollama:11434`). External traffic only reaches ports explicitly mapped: `5432:5432` , `6379:6379` , etc.

12.2 AI Service Dockerfile

```

FROM python:3.11-slim

WORKDIR /app

# Layer caching: copy requirements before code
COPY requirements.txt .

# Split installs by change frequency
RUN pip install --no-cache-dir fastapi uvicorn[standard] pydantic-settings httpx redis
RUN pip install --no-cache-dir qdrant-client "FlagEmbedding>=1.2.0"
RUN pip install --no-cache-dir torch --extra-index-url https://download.pytorch.org/whl/cpu
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Pre-download models during build (cache them in image layers)
RUN python -c "from FlagEmbedding import BGEM3FlagModel; BGEM3FlagModel('BAAI/bge-m3') || true
RUN python -c "from FlagEmbedding import FlagReranker; FlagReranker('BAAI/bge-reranker-v2-m3') || true

EXPOSE 9000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "9000"]

```

12.3 Railway Deployment (Production)

Railway is a cloud platform that builds and deploys Docker containers. Each service in docker-compose becomes a separate Railway service.

Steps to deploy:

1. Create Railway account at railway.app
2. Create new project
3. Add PostgreSQL plugin (managed database)
4. Add Redis plugin (managed cache)
5. Deploy services from GitHub:
 - Connect repo
 - Set `RAILWAY_DOCKERFILE_PATH` for each service
 - Set environment variables from Railway dashboard

Environment variables for production (Railway):

```
# Backend service
DATABASE_URL=postgresql://... # Provided by Railway PostgreSQL plugin
SECRET_KEY=<random 64-char string>
ENVIRONMENT=production
AI_SERVICE_URL=https://ai-service.railway.app
BACKEND_CORS_ORIGINS=https://careerbrownie.com
INTERNAL_API_KEY=<random 32-char string>

# AI service
OLLAMA_URL=<managed Ollama service or Replicate API>
QDRANT_URL=https://qdrant-cluster.cloud.qdrant.io
REDIS_URL=redis://:password@... # Provided by Railway Redis plugin
ANTHROPIC_API_KEY=sk-ant-...

# Frontend (Vercel, not Railway)
NEXT_PUBLIC_API_URL=https://api.careerbrownie.com/api/v1
NEXT_PUBLIC_AI_URL=https://ai.careerbrownie.com
NEXT_PUBLIC_APP_URL=https://careerbrownie.com
```

12.4 Vercel Frontend Deployment

The Next.js frontend is best deployed on Vercel (made by the same team).

```
# Install Vercel CLI
npm install -g vercel

# Deploy from frontend-next directory
cd margveda/frontend-next
vercel --prod

# Set environment variables
vercel env add NEXT_PUBLIC_API_URL production
vercel env add NEXT_PUBLIC_AI_URL production
vercel env add NEXT_PUBLIC_APP_URL production
```

12.5 Production Nginx Configuration

```
server {
    listen 80;
    server_name careerbrownie.com www.careerbrownie.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name careerbrownie.com;

    ssl_certificate /etc/letsencrypt/live/careerbrownie.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/careerbrownie.com/privkey.pem;

    # Rate limiting zones
    limit_req_zone $binary_remote_addr zone=api:10m rate=20r/s;
    limit_req_zone $binary_remote_addr zone=chat:10m rate=5r/s;
    limit_req_zone $binary_remote_addr zone=auth:10m rate=3r/s;

    location / {
        proxy_pass http://frontend:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        limit_req zone=api burst=40 nodelay;
        proxy_pass http://backend:8000;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /api/v1/auth/ {
        limit_req zone=auth burst=6 nodelay;
        proxy_pass http://backend:8000;
    }

    location /api/v2/chat/ {
        limit_req zone=chat burst=10 nodelay;
        proxy_pass http://ai_service:9000;
        proxy_read_timeout 120;
    }
}
```

Part 13: Adding New Features

13.1 Feature Development Framework

When adding any new feature to CareerBrownie, follow this sequence:

1. **Database** — Add/modify model + create Alembic migration
2. **Schema** — Add Pydantic request/response schemas
3. **Service** — Implement business logic
4. **Router** — Add HTTP endpoints
5. **Frontend** — Add API call + UI component
6. **Test** — Write tests for the new feature
7. **Document** — Update OpenAPI docs (auto-generated by FastAPI)

13.2 Example: Adding Stripe Payments

Step 1: Database model

```
# backend/app/models/subscription.py
class Subscription(Base):
    __tablename__ = "subscriptions"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    stripe_customer_id = Column(String(100))
    stripe_subscription_id = Column(String(100))
    plan = Column(String(50)) # "basic", "premium", "enterprise"
    status = Column(String(50)) # "active", "cancelled", "past_due"
    current_period_end = Column(DateTime)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Step 2: Alembic migration

```
alembic revision --autogenerate -m "add_subscriptions_table"
alembic upgrade head
```

Step 3: Router

```
# backend/app/routers/payments.py
import stripe
stripe.api_key = settings.STRIPE_SECRET_KEY

@router.post("/checkout")
async def create_checkout(
    plan: str,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    session = stripe.checkout.Session.create(
        customer_email=current_user.email,
        payment_method_types=["card"],
        line_items=[{"price": PLAN_PRICE_IDS[plan], "quantity": 1}],
        mode="subscription",
        success_url=f"{settings.FRONTEND_URL}/payment/success?session_id={{CHECKOUT_SESSION_ID}}",
        cancel_url=f"{settings.FRONTEND_URL}/pricing",
        metadata={"user_id": current_user.id, "plan": plan},
    )
    return {"checkout_url": session.url}

@router.post("/webhook")
async def stripe_webhook(request: Request):
    payload = await request.body()
    sig = request.headers.get("stripe-signature")
    event = stripe.Webhook.construct_event(payload, sig, settings.STRIPE_WEBHOOK_SECRET)

    if event["type"] == "checkout.session.completed":
        session = event["data"]["object"]
        user_id = int(session["metadata"]["user_id"])
        plan = session["metadata"]["plan"]
        # Update user subscription in DB
        upgrade_user_plan(db, user_id, plan)

    return {"received": True}
```

Step 4: Frontend

```
// src/app/pricing/page.tsx
const handleSubscribe = async (plan: string) => {
    const response = await api.post("/payments/checkout", null, {
        params: { plan }
    });
    window.location.href = response.data.checkout_url;
    // Stripe handles the rest – card input, payment processing
};
```

13.3 Example: Adding Video Calls

For live counselling sessions, integrate a WebRTC service like Daily.co or Agora:

```
# backend/app/routers/sessions.py
import requests

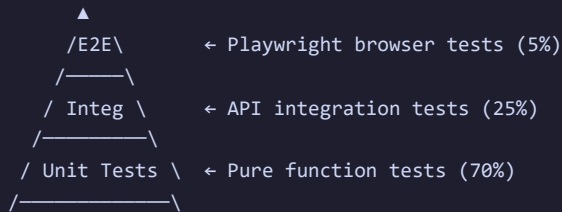
@router.post("/sessions/{session_id}/video")
async def create_video_room(
    session_id: int,
    user: User = Depends(require_roles(["counsellor", "admin"])),
):
    # Create a Daily.co room
    response = requests.post(
        "https://api.daily.co/v1/rooms",
        headers={"Authorization": f"Bearer {settings.DAILY_API_KEY}"},
        json={"properties": {"exp": int(time.time()) + 3600}}, # 1 hour expiry
    )
    room = response.json()

    # Save room URL to session
    session = db.query(CounsellingSession).get(session_id)
    session.video_room_url = room["url"]
    db.commit()

    return {"room_url": room["url"]}
```


Part 14: Testing

14.1 Test Pyramid



14.2 Backend Unit Tests

```

# backend/tests/test_auth.py
import pytest
from app.core.security import hash_password, verify_password, create_access_token, decode_token

def test_password_hashing():
    password = "SecurePassword123!"
    hashed = hash_password(password)

    assert hashed != password # Must not store plain text
    assert verify_password(password, hashed) # Correct password passes
    assert not verify_password("wrongpassword", hashed) # Wrong password fails

def test_jwt_roundtrip():
    payload = {"sub": "42", "role": "student"}
    token = create_access_token(payload)

    decoded = decode_token(token)
    assert decoded["sub"] == "42"
    assert decoded["role"] == "student"
    assert "exp" in decoded # Expiry was added
  
```

14.3 Integration Tests

```
# backend/tests/test_regression.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_register_and_login():
    # Register
    r = client.post("/api/v1/auth/register", json={
        "name": "Test Student",
        "email": "test@example.com",
        "password": "testpass123",
    })
    assert r.status_code == 201
    token = r.json()["access_token"]

    # Use token
    r = client.get("/api/v1/auth/me", headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 200
    assert r.json()["email"] == "test@example.com"

def test_admin_route_requires_admin_role():
    # Login as regular student
    r = client.post("/api/v1/auth/login", json={
        "email": "test@example.com",
        "password": "testpass123",
    })
    token = r.json()["access_token"]

    # Try admin endpoint - should fail
    r = client.get("/api/v1/admin/users",
        headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 403
```

14.4 Running Tests

```
# Backend tests
docker exec careerbrownie_backend pytest tests/ -v

# Specific test file
docker exec careerbrownie_backend pytest tests/test_auth.py -v

# With coverage report
docker exec careerbrownie_backend pytest tests/ --cov=app --cov-report=html
```

Part 15: Security

15.1 OWASP Top 10 Mapping

OWASP Risk	CareerBrownie Mitigation
A01: Broken Access Control	RBAC with <code>require_roles()</code> , JWT validation on every request
A02: Cryptographic Failures	bcrypt for passwords (work factor 12), HS256 JWT signatures
A03: Injection	SQLAlchemy parameterized queries prevent SQL injection
A04: Insecure Design	Separation of AI service (internal only), admin endpoints protected
A05: Security Misconfiguration	Environment variables for all secrets, no hardcoded credentials
A06: Vulnerable Components	Regular <code>pip audit</code> and <code>npm audit</code> dependency scanning
A07: Auth Failures	JWT expiry (24h), rate limiting on auth endpoints (3r/s)
A08: Software Integrity	Docker images from official registries only
A09: Logging Failures	Structured logging on all requests, error tracking
A10: SSRF	AI service internal key prevents external access

15.2 Rate Limiting

```
# backend/app/middleware/rate_limit.py
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@router.post("/auth/login")
@limiter.limit("5/minute") # Max 5 login attempts per minute per IP
async def login(request: Request, ...):
    ...

@router.post("/auth/register")
@limiter.limit("3/minute") # Max 3 registrations per minute per IP
async def register(request: Request, ...):
    ...
```

15.3 Environment Variables Security

Never commit secrets to Git. The `.gitignore` must include:

```
# .gitignore
.env
.env.local
.env.production
.env*.local
margveda/.env
margveda/frontend-next/.env.local
*.pem
*.key
secrets/
```

For production, use a secrets manager (AWS Secrets Manager, HashiCorp Vault) rather than `.env` files.

15.4 SQL Injection Prevention

SQLAlchemy's ORM automatically parameterizes queries:

```
# SAFE – parameterized
user = db.query(User).filter(User.email == email).first()
# Generates: SELECT * FROM users WHERE email = $1 -- [email value]

# DANGEROUS – never do this
db.execute(f"SELECT * FROM users WHERE email = '{email}'")
# Vulnerable to: email = "'; DROP TABLE users; --"
```

Always use the ORM or explicit parameterized queries. Never use f-strings to build SQL.

Part 16: Scaling Strategy

16.1 — 100 Concurrent Users (Current State)

Infrastructure: Single Docker Compose node

Specifications:

- 4 vCPU, 8GB RAM server
- Single PostgreSQL instance
- Single Redis instance
- Ollama with qwen2.5:0.5b

Bottlenecks at this scale:

- LLM inference: 1 request at a time (sequential)
- No connection pooling
- No caching

Monthly cost: ~\$40-60 (Digital Ocean Droplet or Hetzner CX31)

16.2 — 1,000 Concurrent Users

Changes required:

1. Add PgBouncer for connection pooling (1,000 app connections → 25 Postgres connections)
2. Add Redis caching for hot reads (career lists, college lists)
3. Scale FastAPI to 4 Gunicorn workers
4. Add Nginx as reverse proxy with rate limiting

```
# docker-compose addition
pgbouncer:
  image: pgbouncer/pgbouncer:latest
  environment:
    POOL_MODE: transaction
    MAX_CLIENT_CONN: 1000
    DEFAULT_POOL_SIZE: 25
```

```
# Cache frequently-read data
async def get_career_list(db: Session) -> list:
    cached = await cache_get("careers:list")
    if cached:
        return cached
    careers = db.query(Career).filter(Career.is_active == True).all()
    result = [c.to_dict() for c in careers]
    await cache_set("careers:list", result, ttl=600)
    return result
```

Monthly cost: ~\$250

16.3 — 10,000 Concurrent Users

Changes required:

1. Migrate to Kubernetes (AWS EKS or GKE)
2. HorizontalPodAutoscaler (3–20 backend pods)
3. AWS RDS Aurora with read replicas
4. Replace Ollama with vLLM for 10-50x LLM throughput
5. GPU node for AI inference

```
# k8s/backend-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Monthly cost: ~\$1,850

16.4 — 100,000 Concurrent Users

Changes required:

1. Multi-region deployment (India, US, Europe)
2. CloudFront CDN for static assets
3. Database sharding by geography
4. Kafka for event-driven architecture
5. 5x A10G GPUs for inference

Monthly cost: ~\$13,300

16.5 — 1,000,000 Concurrent Users

Changes required:

1. Cloudflare Anycast (edge compute)
2. Kubernetes service mesh (Istio)
3. Fine-tuned Llama 3.1 70B on Indian career data
4. CQRS pattern (write to PostgreSQL, read from Elasticsearch)
5. Feature flags for progressive rollouts

Monthly cost: ~\$149,000

Revenue model: At ₹299/month per paying user, need 50,000 paying users (5% of 1M) = ₹1.5 crore/month gross to sustain costs.

Part 17: Complete User Flow Diagrams

17.1 New Student Onboarding

```

STUDENT ARRIVES → Landing Page (/)
  |
  |— Clicks "Get Started"
  ↓
Register Page
  | Fills: name, email, password, phone
  | Zod validation runs on frontend
  ↓
POST /api/v1/auth/register
  | Check email uniqueness
  | Hash password (bcrypt)
  | INSERT INTO users
  | Create JWT (24h expiry)
  ↓
AuthContext.login(token, user)
  | localStorage.setItem("token")
  | axios header set
  ↓
/dashboard/student
  | First message in chat
  ↓
POST http://localhost:9000/api/v2/chat/
  | Load Redis history (empty)
  | Detect intent
  | Rewrite query
  | Hybrid retrieval (Qdrant)
  | Reranking (bge-reranker)
  | LLM generation (qwen2.5:0.5b)
  | Save to Redis
  ↓
Response with career cards

```

17.2 Lead Capture Flow

```

VISITOR on landing page
  | Clicks "Get Free Counselling"
  ↓
Lead Modal
  | name, email, phone, goal
  ↓
POST /api/v1/leads/
  | Calculate score (10-100)
  | INSERT INTO leads
  | Trigger n8n webhook (async)
  ↓
n8n workflow runs:
  ├── Send welcome SMS (Twilio)
  ├── Add to Google Sheet
  ├── Score > 50? → Hot lead notification to counsellor
  └── Add to email drip campaign
  ↓
"Thanks! We'll call you within 2 hours"

```

17.3 AI RAG Pipeline (Internal)

```

User message: "Best MBA colleges under 5 lakhs?"
  ↓
1. Intent: "college_info"
  ↓
2. Query rewrite (LLM):
  → "affordable MBA colleges India fees"
  → "MBA programs 5 lakh budget"
  → "business school low cost India"
  ↓
3. Hybrid retrieval per query:
  Dense (semantic) + Sparse (BM25) → RRF fusion
  → 30 candidates, deduped to ~20
  ↓
4. Reranking (cross-encoder):
  Score all 20 pairs (query, doc)
  → Top 3 selected
  ↓
5. LLM generation:
  System + context (3 docs) + user message
  → 3-8 second generation
  ↓
6. Response + agent_data (college cards)

```

17.4 Authentication Token Lifecycle

```
LOGIN → JWT created (24h expiry)
↓
Stored in localStorage
↓
Every API call: Authorization: Bearer <token>
↓
Backend validates: signature + expiry
↓
Token expires after 24 hours
↓
Next call returns 401
↓
Axios interceptor catches 401
↓
localStorage.clear() + redirect to /login
```

17.5 Admin Dashboard Flow

```
ADMIN logs in
↓
JWT role = "admin"
↓
Redirect → /admin/dashboard
↓
Parallel API calls:
  GET /admin/stats
  GET /admin/users
  GET /admin/leads
↓
Renders KPIs, user table, lead pipeline
↓
Promote user to counsellor:
  PATCH /admin/users/{id}/role
  Body: {"role": "counsellor"}
↓
Backend: UPDATE users SET role = 'counsellor'
```

Part 18: Code Explanation

18.1 Frontend Files

src/app/layout.tsx — Root Layout

Why it exists: The outermost HTML shell that wraps every page. Loads global fonts, sets default SEO metadata, wraps everything in AuthProvider.

Key function: `RootLayout({ children })`

- Sets `<html lang="en">` for accessibility
- Wraps children in `<AuthProvider>` for global auth state
- Adds `<FloatingChatButton>` so chat is available everywhere
- Exports `metadata` for SEO defaults

Performance: Uses `next/font/google` to self-host Inter font, eliminating external network requests and CLS.

src/contexts/AuthContext.tsx — Authentication State

Why it exists: Centralises auth state so any component can call `useAuth()` without prop drilling.

Inputs: `login(token, user)` and `logout()` called by auth pages and the 401 interceptor.

Outputs: `user`, `token`, `isAuthenticated`, `role` provided to all children via React Context.

On page load: `useEffect` with `[]` reads `localStorage` to restore previous session. This must run in `useEffect` because `localStorage` doesn't exist during server-side rendering.

Security trade-off: Storing JWT in `localStorage` is vulnerable to XSS. The alternative (`httpOnly` cookies) is more secure but requires CSRF protection.

src/lib/api.ts — HTTP Client

Why it exists: Pre-configured Axios instance so all components automatically get auth headers and 401 handling.

Request interceptor: Reads token from `localStorage` and adds `Authorization: Bearer` header to every request.

Response interceptor: Catches 401 responses and redirects to `/login`. This is the "logout" that happens when a JWT expires.

Critical detail: The `return Promise.reject(error)` at the end of the error handler must stay — without it, all API errors become silent successes.

`src/components/chat/ChatWidget.tsx` — Chat Interface

Why it exists: Core product UI — students type questions, see AI responses.

State: `messages[]`, `input`, `isLoading`, `sessionId`, `careerCards[]`

`sendMessage()` : Appends user message to state immediately (optimistic UI), POSTs to AI service, appends AI response, handles errors with fallback message.

Why native `fetch` not `axios`: ChatWidget calls port 9000 (AI service) not port 8000 (backend). Using native fetch avoids the auth interceptor — AI service uses `X-Internal-API-Key` not JWT.

Session ID: Generated once with `crypto.randomUUID()`, stored in `sessionStorage`. Unique per tab, reset when tab closes — correct behavior for conversation context.

`src/components/chat/FloatingChatButton.tsx` — Chat Button

Why `ssr: false` :

```
const ChatWidget = dynamic(() => import("./ChatWidget"), { ssr: false });
```

ChatWidget accesses `sessionStorage` which doesn't exist on the server. `ssr: false` excludes it from SSR entirely, preventing runtime crashes.

Trade-off: Button appears ~100-300ms after page load (client-side only). Acceptable for a chat widget; unacceptable for above-the-fold content.

18.2 Backend Files

`backend/app/main.py` — Application Factory

Why it exists: Creates the FastAPI app, registers middleware and routers, runs startup tasks.

`startup()` event: Calls `Base.metadata.create_all()` — creates tables if they don't exist. Safe to call repeatedly; never drops data.

CORS middleware: Allows the frontend at `localhost:3000` to make API calls to `localhost:8000`. Without this, all browser requests fail with CORS error.

backend/app/core/config.py — Settings

Why it exists: All configuration from environment variables with type validation and defaults.

Settings class (Pydantic): Reads env vars on import. Type mismatches cause `ValidationError` at startup — fail fast, not silently.

settings = Settings() : Singleton — created once on import. Every module that `from app.core.config import settings` gets the same instance.

backend/app/core/security.py — Cryptography

Why it exists: Centralises JWT creation/validation and password hashing. Easy to audit and change in one place.

hash_password() : bcrypt with auto work factor. Takes ~100ms intentionally — makes brute-force attacks computationally infeasible.

create_access_token() : Creates JWT with `sub` (user_id), `role`, and `exp` (expiry) claims. Signs with `SECRET_KEY` using HS256.

decode_token() : Verifies JWT signature and expiry. Raises `HTTPException(401)` on any failure.

backend/app/dependencies/auth.py — Auth Dependencies

Why it exists: Reusable FastAPI dependencies for authentication and authorization.

get_current_user : Extracts user from JWT on every protected request. No database caching — each request re-reads the user from DB to catch deactivations.

require_roles(allowed_roles) : Factory that returns a dependency. The closure captures `allowed_roles` and checks the current user's role.

```
# Usage:
Depends(require_roles(["admin"]))           # Admin only
Depends(require_roles(["counsellor", "admin"])) # Either role
Depends(get_current_user)                   # Any authenticated user
```

backend/app/services/recommendation_service.py — Recommendation Service

Why it exists: Business logic for getting and saving career recommendations. Separates HTTP concerns (router) from business logic.

AIServiceClient : Uses `urllib.request` (Python stdlib) to call AI service. No external HTTP library needed — keeps the backend container lean.

save_recommendation() : Persists AI-generated recommendations to PostgreSQL so users can review their history.

18.3 AI Service Files

`ai_service/app/main.py` — AI App Factory

Why it exists: Creates FastAPI app, runs startup sequence, provides health endpoint.

startup() :

1. `init_embedder()` — loads bge-m3 (2-3 min first run, cached on restart)
2. `ensure_collections()` — creates Qdrant collections if missing
3. `get_llm()` — verifies Ollama is up and model is loaded

Health endpoint: Checks Redis, Qdrant, and LLM connectivity. Used by Docker healthcheck and monitoring.

`ai_service/app/chatbot/engine.py` — RAG Pipeline

Why it exists: Core AI orchestration. Coordinates 9-step pipeline from user message to response.

Pipeline steps:

1. Load Redis conversation history
2. Detect intent (regex, fast)
3. Rewrite query (LLM, slow)
4. Hybrid retrieval (Qdrant, medium)
5. Reranking (cross-encoder, medium)
6. Build context string
7. LLM generation (slow, 3-8 seconds)
8. Save to Redis
9. Extract structured data for frontend

history[-6:] : Only last 6 messages sent to LLM. Keeps prompt size manageable while preserving immediate conversation context.

`ai_service/app/llm/ollama.py` — LLM Client

Why it exists: Abstracts LLM calls behind an interface. Implements Ollama and Anthropic fallback.

OllamaLLM.is_available() : Checks if model is actually loaded, not just if Ollama API responds:

```
models = r.json().get("models", [])
model_base = self._model.split(":")[0]
return any(m.get("name", "").startswith(model_base) for m in models)
```

Without this check, `get_llm()` would return Ollama even with no model loaded, causing cryptic errors.

AnthropicFallback : Calls Claude API when Ollama is unavailable. Requires `ANTHROPIC_API_KEY` with active credits. Falls back gracefully — users get AI responses even if local LLM is down.

get_llm() : Factory that returns the best available LLM. Tries Ollama first, falls back to Anthropic, raises `RuntimeError` if neither is available.

ai_service/app/recommender/career_recommender.py — Career Recommender

Why it exists: Structured recommendation endpoint separate from chat. Takes student profile (interests, education, location, budget) and returns ranked career matches.

calculate_match_score() : Composite score:

- 40% interest overlap (student interests $\cap$ career tags)
- 20% location availability
- 20% education match
- 20% base score

Higher-weight on interests because that's the most predictive factor for career satisfaction.

18.4 Docker Infrastructure Files

docker-compose.yml — Service Orchestration

Key design decisions:

postgres healthcheck: `pg_isready` is a built-in PostgreSQL tool that checks if the server accepts connections. More reliable than `curl` because it speaks the PostgreSQL protocol.

ollama entrypoint: `ollama serve & sleep 8 && ollama pull qwen2.5:0.5b; wait`

- `&` runs server in background
- `sleep 8` waits for server to initialize
- `ollama pull` downloads model
- `; wait` (not `&& wait`) waits for background process even if pull fails

ollama healthcheck: Uses bash TCP redirection (`bash -c '</dev/tcp/localhost/11434'`) because the Ollama image doesn't include `curl` .

depends_on with condition: service_healthy : Docker waits for the dependency to pass its healthcheck before starting the dependent service. Without this, `ai_service` would crash trying to connect to Qdrant before Qdrant is ready.

Named volumes: `postgres_data` , `redis_data` , etc. persist across `docker-compose down` restarts. Only `docker-compose down -v` removes volumes.

ai_service/Dockerfile — AI Service Image

Layer caching strategy: Copy `requirements.txt` before copying source code. Docker caches each layer. If only source code changes (not requirements), Docker skips the `pip install` layers, saving 5-15 minutes on rebuilds.

Split `pip install` by change frequency: Framework packages (fastapi, uvicorn) change rarely. ML packages (torch, FlagEmbedding) change occasionally. Splitting them maximises cache reuse.

`--host 0.0.0.0` : Inside Docker, `localhost` refers to the container itself. `0.0.0.0` listens on all interfaces, making the service accessible from outside the container through the Docker bridge network.

`|| true` on model pre-download: If the model download fails during build (DNS issues, network problems), the build continues. The model downloads at runtime on first use instead. This prevents a temporary network issue from blocking all builds.

Appendix A: Environment Variables Reference

FastAPI Backend

Variable	Default	Required in Prod	Description
PROJECT_NAME	Career Brownie API	No	Shown in Swagger
VERSION	1.0.0	No	API version
ENVIRONMENT	development	No	Affects log level
DATABASE_URL	sqlite:///./careerbrownie.db	Yes	PostgreSQL connection
SECRET_KEY	change-me	Yes	JWT signing key (32+ chars, random)
JWT_ALGORITHM	HS256	No	JWT algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	1440	No	24 hours
AI_SERVICE_URL	http://localhost:9000	Yes	http://ai_service:9000 in Docker
INTERNAL_API_KEY	dev-internal-key	Yes	Shared secret with AI service
BACKEND_CORS_ORIGINS	http://localhost:3000	Yes	Comma-separated frontend URLs

AI Service

Variable	Default	Required in Prod	Description
OLLAMA_URL	http://localhost:11434	No	http://ollama:11434 in Docker
OLLAMA_MODEL	qwen2.5:0.5b	No	LLM model name
OLLAMA_TIMEOUT	120	No	Request timeout seconds
OLLAMA_TEMPERATURE	0.7	No	0=deterministic, 1=creative
OLLAMA_MAX_TOKENS	1024	No	Max response tokens
QDRANT_URL	http://localhost:6333	No	http://qdrant:6333 in Docker
REDIS_URL	redis://localhost:6379/1	No	Redis connection string
INTERNAL_API_KEY	dev-internal-key	Yes	Must match backend key

<code>ANTHROPIC_API_KEY</code>	<code>''</code>	No	Claude API key (fallback LLM)
<code>EMBED_DEVICE</code>	<code>cpu</code>	No	<code>cuda</code> for GPU

Frontend (Next.js)

Variable	Description
<code>NEXT_PUBLIC_API_URL</code>	Backend URL: <code>http://localhost:8000/api/v1</code>
<code>NEXT_PUBLIC_AI_URL</code>	AI service URL: <code>http://localhost:9000</code>
<code>NEXT_PUBLIC_APP_URL</code>	Frontend URL: <code>http://localhost:3000</code>

Appendix B: Common Commands Quick Reference

```
# — START EVERYTHING —————
cd margveda
docker-compose up -d postgres redis qdrant ollama
# Wait 60s for Ollama model pull, then:
docker-compose up -d ai_service backend
cd frontend-next && npm run dev

# — CHECK STATUS —————
docker ps
docker logs careerbrownie_ollama --tail 50
curl -s http://localhost:9000/health | python -m json.tool
curl -s http://localhost:8000/health

# — TEST AI CHAT —————
curl -s -X POST http://localhost:9000/api/v2/chat/ \
  -H "Content-Type: application/json" \
  -d '{"message":"What career suits someone who loves math?","session_id":"test-1"}'

# — DATABASE —————
# Connect
docker exec -it careerbrownie_postgres psql -U careerbrownie -d careerbrownie
# Backup
docker exec careerbrownie_postgres pg_dump -U careerbrownie careerbrownie > backup.sql
# Restore
docker exec -i careerbrownie_postgres psql -U careerbrownie careerbrownie < backup.sql
# Run migrations
docker exec careerbrownie_backend alembic upgrade head

# — OLLAMA —————
docker exec careerbrownie_ollama ollama list
docker exec careerbrownie_ollama ollama pull qwen2.5:0.5b

# — REBUILD —————
docker-compose up -d --build ai_service
docker-compose up -d --build backend
```

Appendix C: Troubleshooting

Container exits immediately

```
docker logs careerbrownie_<service> --tail 100
# Look for: ValidationError (missing env var), connection refused (dependency not ready)
```

AI service unhealthy

```
docker exec careerbrownie_ai_service curl http://ollama:11434/api/tags
docker exec careerbrownie_ollama ollama list
# If model missing:
docker exec careerbrownie_ollama ollama pull qwen2.5:0.5b
```

Chat returns error message

```
docker logs careerbrownie_ai_service --tail 50
curl http://localhost:9000/health
docker exec careerbrownie_ollama ollama list
```

Login returns 401

```
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "test@example.com", "password": "testpass"}'
# Check: user exists, password correct, backend running
```

Qdrant returns 0 results

The vector database is empty. Run the ingestion script:

```
cd margveda  
python scripts/ingest_careers.py  
python scripts/ingest_colleges.py
```

End of CareerBrownie Software Engineering Handbook v1.0 Generated: June 2026